



AuthiChain Production Deployment Checklist

Version: 1.0.0-phase1

Last Updated: October 18, 2025

Deployment Target: Production



Pre-Deployment Requirements

1. Infrastructure Setup

Hosting Platform (Choose One)

Option A: Vercel (Recommended)

- ☐ Sign up for Vercel account
- ☐ Install Vercel CLI: `npm i -g vercel`
- ☐ Connect GitHub repository
- ☐ Configure custom domain

Option B: Railway

- ☐ Sign up for Railway account
- ☐ Install Railway CLI
- ☐ Create new project
- ☐ Connect repository

Option C: AWS

- ☐ EC2 instance (t3.medium or better)
- ☐ RDS PostgreSQL instance
- ☐ S3 bucket for static assets
- ☐ CloudFront CDN
- ☐ Route 53 for DNS

Database

PostgreSQL Requirements:

- ☐ PostgreSQL 14+ installed
- ☐ Database created
- ☐ User with full permissions
- ☐ SSL enabled (production)
- ☐ Backup schedule configured

Recommended Providers:

- ☒ Railway Database (easiest)
- ☒ Supabase
- ☒ AWS RDS
- ☒ DigitalOcean Managed Database

CDN & Static Assets

- ☐ Cloudflare account setup

- [] Domain added to Cloudflare
- [] SSL certificate configured
- [] Page rules configured
- [] Caching rules set

Environment Configuration

Required Environment Variables

Create `.env.production` file with these variables:

```

# =====
# DATABASE
# =====
DATABASE_URL="postgresql://username:password@host:5432/database?sslmode=require"

# =====
# AUTHENTICATION
# =====
# Generate with: openssl rand -base64 32
NEXTAUTH_SECRET="your-32-char-secret-key-here"
NEXTAUTH_URL="https://yourdomain.com"

# =====
# STRIPE PAYMENTS (Live Keys)
# =====
STRIPE_SECRET_KEY="sk_live_XXXXXXXXXXXX"
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY="pk_live_XXXXXXXXXXXX"
STRIPE_WEBHOOK_SECRET="whsec_XXXXXXXXXXXX"

# Stripe Price IDs (Production)
STRIPE_STARTER_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_STARTER_MONTHLY_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_STARTER_ANNUAL_PRICE_ID="price_XXXXXXXXXXXX"

STRIPE_CREATOR_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_CREATOR_MONTHLY_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_CREATOR_ANNUAL_PRICE_ID="price_XXXXXXXXXXXX"

STRIPE_PRO_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_PRO_MONTHLY_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_PRO_ANNUAL_PRICE_ID="price_XXXXXXXXXXXX"

STRIPE_ENTERPRISE_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_ENTERPRISE_MONTHLY_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_ENTERPRISE_ANNUAL_PRICE_ID="price_XXXXXXXXXXXX"

STRIPE_BRAND_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_BRAND_MONTHLY_PRICE_ID="price_XXXXXXXXXXXX"
STRIPE_BRAND_ANNUAL_PRICE_ID="price_XXXXXXXXXXXX"

# =====
# IPFS / NFT STORAGE
# =====
# Get key from: https://nft.storage/manage/
NFT_STORAGE_API_KEY="eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."

# =====
# SITE CONFIGURATION
# =====
NEXT_PUBLIC_SITE_URL="https://yourdomain.com"
NEXT_PUBLIC_APP_NAME="AuthiChain"
NEXT_PUBLIC_APP_VERSION="1.0.0"

# =====
# EMAIL (Optional - for notifications)
# =====
SMTP_HOST="smtp.sendgrid.net"
SMTP_PORT="587"
SMTP_USER="apikey"
SMTP_PASS="SG.XXXXXXXXXXXXXX"
SMTP_FROM="noreply@yourdomain.com"

```

```
# =====
# ANALYTICS (Optional)
# =====
NEXT_PUBLIC_GA_ID="G-XXXXXXXXXX"
NEXT_PUBLIC_GOOGLE_ANALYTICS_ID="UA-XXXXXXXX-1"

# =====
# MONITORING (Optional)
# =====
SENTRY_DSN="https://xxxxx@sentry.io/xxxxx"
SENTRY_AUTH_TOKEN="xxxxxxxxxxxxxx"

# =====
# SECURITY
# =====
CORS_ORIGIN="https://yourdomain.com"
RATE_LIMIT_ENABLED="true"
RATE_LIMIT_MAX_REQUESTS="100"
RATE_LIMIT_WINDOW_MS="900000"

# =====
# BLOCKCHAIN (Phase 2)
# =====
# WEB3_PROVIDER_URL="https://mainnet.infura.io/v3/xxxxx"
# CONTRACT_ADDRESS="0xYourContractAddress"
# PRIVATE_KEY="xxxxxxxxxxxxxx" # Never commit this!
```

Environment Variable Checklist

Critical (Required):

- [] DATABASE_URL - PostgreSQL connection string
- [] NEXTAUTH_SECRET - Authentication secret (32+ chars)
- [] NEXTAUTH_URL - Production URL
- [] STRIPE_SECRET_KEY - Live Stripe secret key
- [] NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY - Live Stripe public key
- [] STRIPE_WEBHOOK_SECRET - Stripe webhook signing secret
- [] NFT_STORAGE_API_KEY - IPFS upload key
- [] NEXT_PUBLIC_SITE_URL - Production URL

Stripe Price IDs (15 total):

- [] All 5 tiers configured (Starter, Creator, Pro, Enterprise, Brand)
- [] Monthly and annual variants for each tier
- [] Price IDs copied from Stripe Dashboard

Optional but Recommended:

- [] Email configuration (SMTP)
 - [] Analytics (Google Analytics)
 - [] Monitoring (Sentry)
 - [] Rate limiting config
-

Database Setup

1. Database Migration

```
# Navigate to app directory
cd /home/ubuntu/authichain_premium/app

# Set production DATABASE_URL
export DATABASE_URL="postgresql://..."

# Run migrations
npx prisma migrate deploy

# Generate Prisma client
npx prisma generate

# Verify connection
npx prisma db push
```

2. Seed Data (Optional)

```
# Seed initial data
npm run prisma:seed
```

3. Database Indexes

Ensure these indexes exist for performance:

```
-- NFT indexes
CREATE INDEX idx_nft_status ON "NFT"(status);
CREATE INDEX idx_nft_owner ON "NFT"("ownerId");
CREATE INDEX idx_nft_creator ON "NFT"("creatorId");
CREATE INDEX idx_nft_collection ON "NFT"("collectionId");
CREATE INDEX idx_nft_created ON "NFT"("createdAt");

-- Collection indexes
CREATE INDEX idx_collection_creator ON "Collection"("creatorId");
CREATE INDEX idx_collection_slug ON "Collection"(slug);

-- Auction indexes
CREATE INDEX idx_auction_status ON "Auction"(status);
CREATE INDEX idx_auction_nft ON "Auction"("nftId");
CREATE INDEX idx_auction_end ON "Auction"("endTime");

-- Bid indexes
CREATE INDEX idx_bid_auction ON "Bid"("auctionId");
CREATE INDEX idx_bid_bidder ON "Bid"("bidderId");
CREATE INDEX idx_bid_created ON "Bid"("createdAt");

-- Offer indexes
CREATE INDEX idx_offer_nft ON "Offer"("nftId");
CREATE INDEX idx_offer_from ON "Offer"("fromUserId");
CREATE INDEX idx_offer_to ON "Offer"("toUserId");
CREATE INDEX idx_offer_status ON "Offer"(status);
```

4. Database Backup

```
# Setup daily backups
pg_dump -h host -U user -d database > backup_$(date +%Y%m%d).sql

# Or use managed backup service
# Railway: Automatic backups enabled
# AWS RDS: Enable automated backups (retention: 7-30 days)
```

Stripe Configuration

1. Create Live Mode Products

Dashboard: <https://dashboard.stripe.com/products>

For each tier, create:

1. Product with name and description
2. Monthly price
3. Annual price (with discount)
4. Copy Price IDs to environment variables

5 Tiers:

- [] Starter (\$97/month, \$970/year)
- [] Creator (\$297/month, \$2,970/year)
- [] Pro (\$697/month, \$6,970/year)
- [] Enterprise (\$1,497/month, \$14,970/year)
- [] Brand (\$1,997/month, \$19,970/year)

2. Configure Webhooks

Webhook URL: `https://yourdomain.com/api/webhooks/stripe`

Events to Subscribe:

- [] `checkout.session.completed`
- [] `customer.subscription.created`
- [] `customer.subscription.updated`
- [] `customer.subscription.deleted`
- [] `invoice.payment_succeeded`
- [] `invoice.payment_failed`

Copy webhook signing secret to `STRIPE_WEBHOOK_SECRET`

3. Test Webhook

```
# Install Stripe CLI
stripe listen --forward-to localhost:3000/api/webhooks/stripe

# Trigger test event
stripe trigger checkout.session.completed
```

IPFS / NFT.Storage Setup

1. Get API Key

1. Go to <https://nft.storage/>
2. Sign up / Login
3. Navigate to "API Keys"
4. Create new API key
5. Copy key to `NFT_STORAGE_API_KEY`

2. Verify IPFS Upload

```
# Test IPFS upload (in Node.js)
curl -X POST http://yourdomain.com/api/upload/ipfs \
  -H "Authorization: Bearer YOUR_SESSION_TOKEN" \
  -F "file=@test-image.png"
```

3. Configure Gateway

Default gateway: `https://nftstorage.link/ipfs/{CID}`

Alternative gateways:

- `https://ipfs.io/ipfs/{CID}`
- `https://cloudflare-ipfs.com/ipfs/{CID}`
- `https://gateway.pinata.cloud/ipfs/{CID}`

Build & Deploy

Option A: Vercel Deployment

```
# Install Vercel CLI
npm i -g vercel

# Login
vercel login

# Deploy
cd /home/ubuntu/authichain_premium/app
vercel --prod

# Configure environment variables in Vercel Dashboard
# Project Settings > Environment Variables
```

Vercel Configuration (`vercel.json`):

```
{
  "buildCommand": "npm run build",
  "devCommand": "npm run dev",
  "installCommand": "npm install",
  "framework": "nextjs",
  "regions": ["iad1"],
  "env": {
    "DATABASE_URL": "@database-url",
    "NEXTAUTH_SECRET": "@nextauth-secret"
  }
}
```

Option B: Railway Deployment

```
# Install Railway CLI
npm i -g @railway/cli

# Login
railway login

# Initialize project
railway init

# Link to project
railway link

# Add environment variables
railway variables set DATABASE_URL="postgresql://..."

# Deploy
railway up
```

Option C: Docker Deployment

Dockerfile:

```
FROM node:18-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

FROM node:18-alpine AS runner

WORKDIR /app
ENV NODE_ENV production

COPY --from=builder /app/package*.json ./
COPY --from=builder /app/.next ./.next
COPY --from=builder /app/public ./public
COPY --from=builder /app/node_modules ./node_modules

EXPOSE 3000
CMD ["npm", "start"]
```

Deploy:


```
# Build image
docker build -t authichain .

# Run container
docker run -p 3000:3000 --env-file .env.production authichain
```

Security Checklist

SSL/TLS Configuration

- ☐ SSL certificate installed (Let's Encrypt or Cloudflare)
- ☐ HTTPS enforced (redirect HTTP to HTTPS)
- ☐ HSTS header enabled
- ☐ Secure cookies configured

Security Headers

Add to `next.config.js` :

```
module.exports = {
  async headers() {
    return [
      {
        source: '/*',
        headers: [
          {
            key: 'X-Frame-Options',
            value: 'DENY',
          },
          {
            key: 'X-Content-Type-Options',
            value: 'nosniff',
          },
          {
            key: 'X-XSS-Protection',
            value: '1; mode=block',
          },
          {
            key: 'Referrer-Policy',
            value: 'strict-origin-when-cross-origin',
          },
          {
            key: 'Permissions-Policy',
            value: 'camera=(), microphone=(), geolocation=()',
          },
        ],
      },
    ];
  },
};
```

Environment Security

- ☐ `.env` files not committed to Git
- ☐ Secrets stored in platform environment variables

- ☐ API keys rotated regularly
- ☐ Database credentials secured
- ☐ Private keys never exposed

Application Security

- ☐ Rate limiting enabled
- ☐ CORS configured for production domain
- ☐ SQL injection prevention (Prisma ORM)
- ☐ XSS protection (React escaping)
- ☐ CSRF tokens implemented

Domain & DNS Configuration

1. Domain Registration

- ☐ Register domain (GoDaddy, Namecheap, Google Domains)
- ☐ Verify ownership
- ☐ Enable domain privacy protection

2. DNS Configuration

Cloudflare DNS (Recommended):

Type	Name	Value	Proxy
A	@	<server-ip>	Proxied
A	www	<server-ip>	Proxied
CNAME	api	yourdomain.com	Proxied
CNAME	cdn	yourdomain.com	Proxied
TXT	@	"v=spf1 include:_spf.google.com ~all"	

Or Point to Vercel:

Type	Name	Value
A	@	76.76.21.21
CNAME	www	cname.vercel-dns.com

3. Email Configuration (Optional)

- ☐ Setup MX records for email
- ☐ Configure SPF record
- ☐ Configure DKIM
- ☐ Configure DMARC



Monitoring & Analytics

1. Error Tracking (Sentry)

```
# Install Sentry
npm install @sentry/nextjs

# Initialize
npx @sentry/wizard -i nextjs

# Configure in next.config.js
```

2. Performance Monitoring

- [] Enable Vercel Analytics (if using Vercel)
- [] Setup Google Analytics
- [] Configure performance budgets
- [] Monitor Core Web Vitals

3. Logging

```
# Production logging
- Use structured logging (Winston, Pino)
- Log errors to external service (Papertrail, Loggly)
- Setup log rotation
- Monitor error rates
```

4. Uptime Monitoring

- [] Setup uptime monitoring (UptimeRobot, Pingdom)
 - [] Configure alerts (email, Slack)
 - [] Monitor critical endpoints
 - [] Setup status page
-



Pre-Launch Testing

Functionality Testing

- [] User registration/login works
- [] MetaMask wallet connection works
- [] Stripe checkout completes successfully
- [] NFT minting works (single & batch)
- [] IPFS upload successful
- [] Collection creation works
- [] Auction creation and bidding works
- [] Offer system functions properly
- [] NFT transfer completes
- [] Dashboard displays correctly

Performance Testing

```
# Load testing with Apache Bench
ab -n 1000 -c 100 https://yourdomain.com/

# Or use k6
k6 run load-test.js
```

- ☐ Homepage loads < 2 seconds
- ☐ API responses < 500ms
- ☐ Images optimized and compressed
- ☐ Database queries optimized
- ☐ CDN caching configured

Security Testing

- ☐ SQL injection testing
- ☐ XSS vulnerability testing
- ☐ CSRF protection verified
- ☐ Authentication bypass testing
- ☐ Rate limiting tested

Browser Compatibility

- ☐ Chrome (latest)
- ☐ Firefox (latest)
- ☐ Safari (latest)
- ☐ Edge (latest)
- ☐ Mobile browsers (iOS Safari, Chrome Mobile)

Mobile Responsiveness

- ☐ iPhone (various models)
- ☐ Android (various models)
- ☐ Tablet (iPad, Android tablets)
- ☐ Desktop (1920x1080, 1366x768, 1024x768)



PWA Configuration

Service Worker

- ☐ Service worker registered
- ☐ Offline page configured
- ☐ Cache strategy implemented
- ☐ Push notifications enabled

Manifest

- ☐ Icons (72x72, 96x96, 128x128, 192x192, 512x512)
- ☐ App name and description
- ☐ Theme color

- ☐ Start URL
- ☐ Display mode (standalone)

Testing PWA

```
# Lighthouse PWA audit  
npx lighthouse https://yourdomain.com --view
```

Score targets:

- ☐ Performance: 90+
- ☐ Accessibility: 90+
- ☐ Best Practices: 90+
- ☐ SEO: 90+
- ☐ PWA: 100



Launch Day Checklist

1 Hour Before Launch

- ☐ Final database backup
- ☐ Verify all environment variables
- ☐ Test payment processing (real transaction)
- ☐ Verify SSL certificate
- ☐ Check monitoring dashboards
- ☐ Prepare rollback plan

At Launch

- ☐ Deploy to production
- ☐ Verify DNS propagation
- ☐ Test critical user flows
- ☐ Monitor error logs
- ☐ Watch server metrics
- ☐ Be available for issues

1 Hour After Launch

- ☐ Check error rates
 - ☐ Verify payments working
 - ☐ Monitor user signups
 - ☐ Check performance metrics
 - ☐ Review user feedback
 - ☐ Document any issues
-

Post-Launch Monitoring

Daily

- ☐ Check error logs
- ☐ Review performance metrics
- ☐ Monitor uptime
- ☐ Verify backup completion
- ☐ Check payment processing

Weekly

- ☐ Review user feedback
- ☐ Analyze usage patterns
- ☐ Update dependencies
- ☐ Security updates
- ☐ Performance optimization

Monthly

- ☐ Full security audit
 - ☐ Database optimization
 - ☐ Cost analysis
 - ☐ Feature usage review
 - ☐ User satisfaction survey
-

Rollback Plan

If Issues Occur

1. Identify the issue

```
bash
# Check logs
vercel logs
# or
railway logs
```

2. Rollback deployment

```
``bash
# Vercel
vercel rollback
```

```
# Railway
railway rollback
```

```
# Docker
docker stop authichain
```

docker run previous-version

...

1. **Restore database** (if needed)

bash

```
psql -h host -U user -d database < backup_previous.sql
```

2. **Notify users**

- Update status page
- Send email notification
- Post on social media

Emergency Contacts

Technical Support

- **Platform:** [Vercel Support](https://vercel.com/support) (https://vercel.com/support)
- **Database:** [Railway Support](https://railway.app/help) (https://railway.app/help)
- **Stripe:** support@stripe.com
- **NFT.Storage:** support@nft.storage

On-Call Engineers

- **Primary:** Your email/phone
- **Secondary:** Team member
- **Database:** DBA contact

Final Pre-Launch Checklist

Critical Items

- [] Production database configured and backed up
- [] All environment variables set
- [] Stripe live keys configured
- [] NFT.Storage API key configured
- [] Domain DNS configured
- [] SSL certificate installed
- [] Monitoring and alerts configured
- [] Error tracking enabled
- [] All tests passing
- [] Performance acceptable
- [] Security audit complete
- [] Team notified of launch
- [] Rollback plan documented
- [] Support team briefed

Ready to Launch?

If all items are checked ☒, you are ready to deploy to production!

Post-Deployment

Success Criteria

- ☐ Website accessible at domain
 - ☐ All pages loading correctly
 - ☐ Payment processing working
 - ☐ NFT minting functional
 - ☐ No critical errors in logs
 - ☐ Performance within targets
 - ☐ Users can sign up and login
 - ☐ Monitoring dashboards green
-

Deployment Guide Version: 1.0.0

Last Updated: October 18, 2025

Next Review: After successful deployment

Questions? Contact: support@authichain.com